115. Distinct Subsequences

Given two strings s and t, return *the number of distinct **subsequences** of* s *which equals* t.

The test cases are generated so that the answer fits on a 32-bit signed integer.

Example 1:

```
Input: s = "rabbbit", t = "rabbit"
Output: 3
Explanation:
As shown below, there are 3 ways you can generate "rabbit" from s.
rabbbit
rabbbit
rabbbit
```

Example 2:

```
Input: s = "babgbag", t = "bag"
Output: 5
Explanation:
As shown below, there are 5 ways you can generate "bag" from s.
babgbag
babgbag
babgbag
babgbag
babgbag
```

Constraints:

- `1 <= s.length, t.length <= 1000`
- `s` and `t` consist of English letters.

Recursive

```
class Solution:
def numDistinct(self, s: str, t: str) -> int:
    if t=="":
        return 1

    elif s == "":
        return 0

    if s[-1] == t[-1]:
        res = self.numDistinct(s[:-1], t[:-1]) + self.numDistinct(s[:-1], t)
    else:
        res = self.numDistinct(s[:-1], t)

    return res
```

DP top down:

```
class Solution:
def __init__(self):
    self.dp = [[-1 for j in range(1001)] for i in range(1001)]
def numDistinct(self, s: str, t: str) -> int:
    m, n = len(s), len(t)
    if n==0:
            self.dp[m-1][n-1] = 1
            return 1
    elif m==0:
            self.dp[m-1][n-1] = 0
            return 0

    if self.dp[m-1][n-1] != -1:
            return self.dp[m-1][n-1]
    if s[-1] == t[-1]:
            self.dp[m-1][n-1] = self.numDistinct(s[:-1], t[:-1]) + self.numDistinc
t(s[:-1], t)
    else:
            self.dp[m-1][n-1] = self.numDistinct(s[:-1], t)

    return self.dp[m-1][n-1]
```

DP bottom up:

```
class Solution:

def numDistinct(self, s: str, t: str) -> int:
            m = len(s)
            n = len(t)
            dp =  [[0 for j in range(n+1)] for i in range(m+1)]

            for i in range(m):
                for j in range(n):
                    if  j ==0:
                        dp[i][j] = 1
                    elif i==0:
                        dp[i][j] = 0
                    elif  s[i] == t[j]:
                        dp[i][j] = dp[i-1][j-1]) + dp[i-1][j])
                    else:
                        dp[i][j] = dp[i-1][j])

    return dp[m][n]
```

DFS (linear Space) :

```
class Solution:

    def numDistinct(self, s: str, t: str) -> int:
        m = len(s)
        n = len(t)
        dp = {}

        def dfs(p, q):
            if q == 0:
                return 1
            elif p == 0:
                return 0
            if (p,q) in dp:
                return dp[(p,q)]
            elif s[p-1] == t[q-1]:
                dp[(p,q)] = dfs(p-1, q-1) + dfs(p-1, q)
            else:
                dp[(p,q)] = dfs(p-1, q)
            return dp[(p,q)]

        return dfs(m,n)
```

207. Course Schedule

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [$a_i$, $b_i$]` indicates that you **must** take course `$b_i$` first if you want to take course `$a_i$`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

```
Input: numCourses = 2, prerequisites = [[1,0]]
Output: true
Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0. So it is possible.
```

Example 2:

```
Input: numCourses = 2, prerequisites = [[1,0],[0,1]]
Output: false
Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0, and to take course 0 you should
```

Constraints:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= 5000`
- `prerequisites[i].length == 2`
- `0 <= $a_i$, $b_i$ < numCourses`
- All the pairs prerequisites[i] are **unique**.

Simple topological sort using indegrees:

```
class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        N = numCourses
        adj = {n:[] for n in range(N)}
        indegree = [0]*N
        for c,p in prerequisites:
            adj[p].append(c)
            indegree[c]+=1

        q = []
        for n in range(N):
            if indegree[n]==0:
                q.append(n)

        completed = 0
        while q:
            # p = q.pop(0) # using queue
            p = q.pop() # using stack
            for c in adj[p]:
                indegree[c]-=1
                if indegree[c]==0:
                    q.append(c)
            completed += 1

        return completed == numCourses
```

try this later:

```
class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        N = numCourses
        adj = {n:[] for n in range(N)}
        for c,p in prerequisites:
            adj[c].append(p)

        visited = [False for i in range(N)]
        ordering = [-1 for i in range(N)]
        i=N-1

        def dfs(i, k):
            visited[k] = True
            for p in adj[k]:
                if visited[p]==False:
                    i = dfs(i,p)
                if p in ordering:
                    return -1
                if i == -1:
                    return -1

            ordering[i] = k
            return i-1

        for k in range(N):
            if visited[k]==False:
                i = dfs (i, k)
                if i == -1:
                    return False
        return True
```

210. Course Schedule II

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [$a_i$, $b_i$]` indicates that you **must** take course `$b_i$` first if you want to take course `$a_i$`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return *the ordering of courses you should take to finish all courses*. If there are many valid answers, return **any** of them. If it is impossible to finish all courses, return **an empty array**.

Example 1:

```
Input: numCourses = 2, prerequisites = [[1,0]]
Output: [0,1]
Explanation: There are a total of 2 courses to take. To take course 1 you should hav
```

Example 2:

```
Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]
Output: [0,2,1,3]
Explanation: There are a total of 4 courses to take. To take course 3 you should hav
So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].
```

Example 3:

```
Input: numCourses = 1, prerequisites = []
Output: [0]
```

Constraints:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= numCourses * (numCourses - 1)`
- `prerequisites[i].length == 2`
- `0 <= a_i, b_i < numCourses`
- `a_i != b_i`
- All the pairs `[a_i, b_i]` are **distinct**.

---

topological sort with ordering using indegrees

```
class Solution:
    def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[i
nt]:
        N = numCourses
        indegree = [0]*N
        adj = {n:[] for n in range(N)}

        for c,p in prerequisites:
            adj[p].append(c)
            indegree[c]+=1

        que = []
        for n in range(N):
            if indegree[n]==0:
                que.append(n)

        ordering = []

        while que:
            p = que.pop(0)
            ordering.append(p)
            for c in adj[p]:
                indegree[c]-=1
                if indegree[c]==0:
                    que.append(c)


        if len(ordering) == N:
            return ordering
        else:
            return []
```

Foreign dictionary question: link: https://neetcode.io/problems/foreign-dictionary (https://neetcode.io/problems/foreign-dictionary)

```
class Solution:
    def foreignDictionary(self, words: List[str]) -> str:
        adj = {c:set() for w in words for c in w}
        N = len(adj)
        indegree = {c:0 for c in adj}

        for i in range(len(words)-1):
            w1, w2 = words[i], words[i+1]
            minlen = min(len(w1), len(w2))
            if len(w1) > len(w2) and w1[:minlen]==w2[:minlen]:
                return ""
            else:
                for j in range(minlen):
                    if w1[j]!=w2[j]:
                        pre = w1[j]
                        post = w2[j]
                        if post not in adj[pre]:
                            indegree[post] +=1
                        adj[pre].add(post)
                        break

        q = []

        for l in indegree:
            if indegree[l] == 0:
                q.append(l)

        print(indegree)
        print(q)
        print(adj)
        alpha = ""

        while q:
            pre = q.pop(0)
            alpha += pre

            for post in adj[pre]:
                indegree[post]-=1
                if indegree[post] == 0:
                    q.append(post)

        print(alpha)
        print(indegree)

        if len(alpha) == len(indegree):
            return alpha
        else:
            return ""
```